CSEN604: Data Bases II

Mohamed Karam Gabr

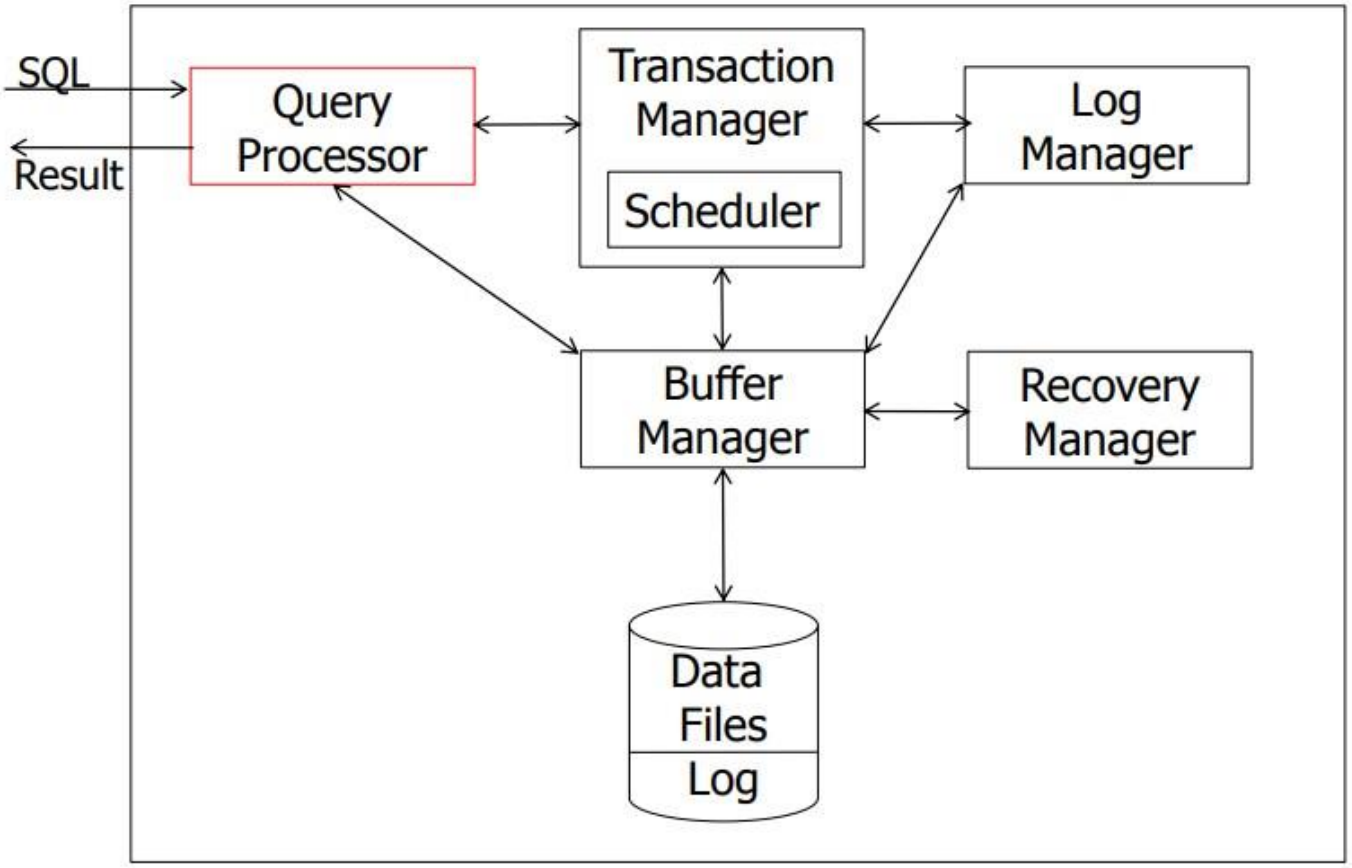
Cost of Physical Plan

Spring 2025

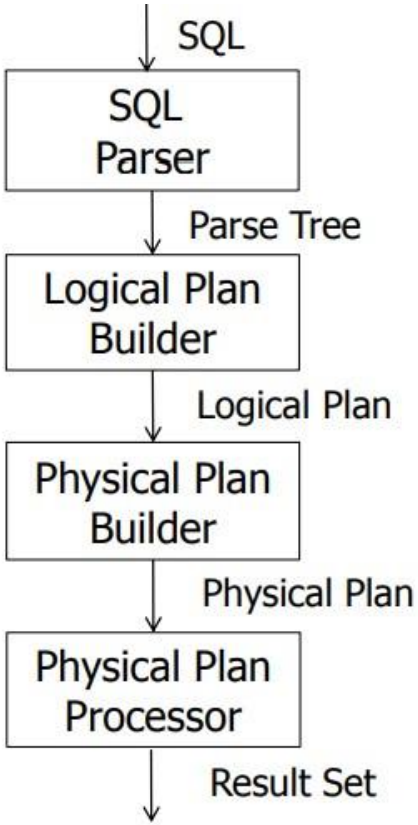
Outline

- Calculating the cost of Physical plan.

Introduction

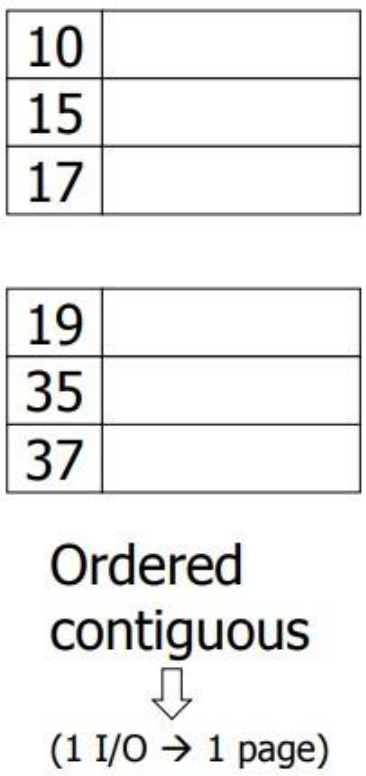


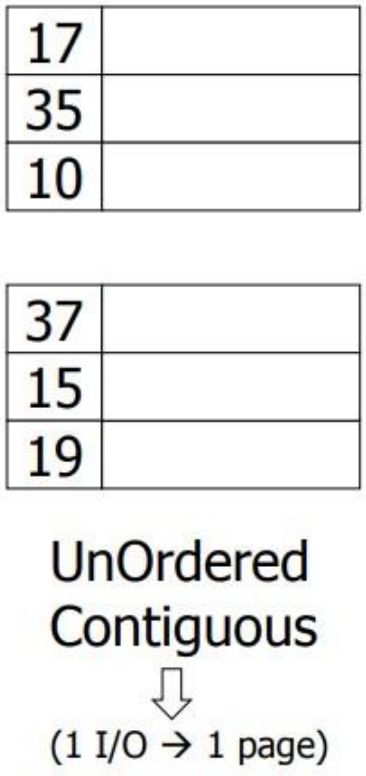
Introduction

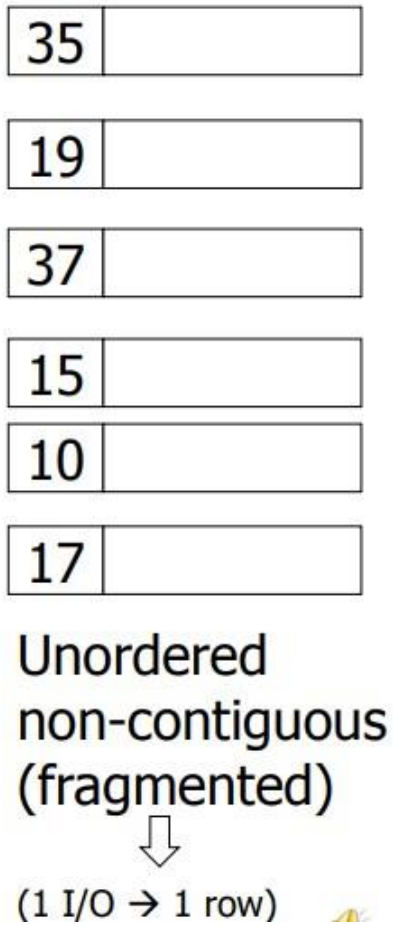


I/O

- 1 I/O is what all the read heads can read without moving to another location on the platters.
- The page rows need to be stored below each other in the same cylinder to be read in 1 I/O.







I/O parameters

- To estimate costs, we have additional parameters:

- B(R) = of blocks containing R tuples.
- f(R) = max of tuples of R per block.
- M = memory blocks available.

We are ignoring Other Cost Centers:
- buffering policies.
- CPU costs.
- I/O for writing the result.

Example $R1 \bowtie R2$ over common attribute C

T(R1) = 10,000
T(R2) = 5,000
S(R1) = S(R2) = 1/10 block [i.e. f(R)=10]
Memory available (M) = 101 blocks

Tables Join

- Transformations: $R1 \bowtie R2$, $R2 \bowtie R1$
- Join algorithms:
  - Iteration (nested loops/aka iterators)
  - Merge join
  - Join with index
  - Hash join

Iteration Join

- Iteration join (conceptually)

```
for each r ∈ R1 do
    for each s ∈ R2 do
        if r.C = s.C then output r,s pair
```

- Too expensive!

Example 1(a) Iteration Join R1 ⋈ R2

- Relations not contiguous
- Recall
  - T(R1) = 10,000 T(R2) = 5,000
  - S(R1) = S(R2) =1/10 block
  - MEM=101 blocks

Cost: for each R1 tuple:

[Read tuple + Read R2]

Total =10,000 [1+5000]=50,010,000 IOs

Iteration Join

- Can we do better?

Use the memory!

(1) Read 100 blocks of R1

(2) Read all of R2 (using 1 block) + join

(3) Repeat until done

- Recall T(R1) = 10,000 T(R2) = 5,000
  S(R1) = S(R2) =1/10 block
  B(R1) = 1000 B(R2) = 500
  MEM=101 blocks

Cost: for each R1 chunk:

Read chunk: 1000 IOs (100 blocks)

Read R2: 5000 IOs (500 blocks)

6000

$$\text{Total} = \frac{10{,}000}{1{,}000} \times 6000 = 60{,}000 \text{ IOs}$$

Iteration Join

- Recall T(R1) = 10,000 T(R2) = 5,000
  S(R1) = S(R2) =1/10 block
  B(R1) = 1000 B(R2) = 500
  MEM=101 blocks

- Can we do better?

  ☛ Reverse join order: R2 ⋈ R1

for each R2 chunk:

Read R2 chunk:	1000	IOs	(100 blocks)
Read R1:	10000	IOs	(1000 blocks)
	11000		

$$\text{Total} = \frac{5000}{1000} \times (1000 + 10{,}000)$$

$$= 55{,}000 \text{ IOs}$$

Iteration Join

Example 1(b) Iteration Join R2 ⋈ R1

- Relations contiguous

Cost

For each R2 chunk:

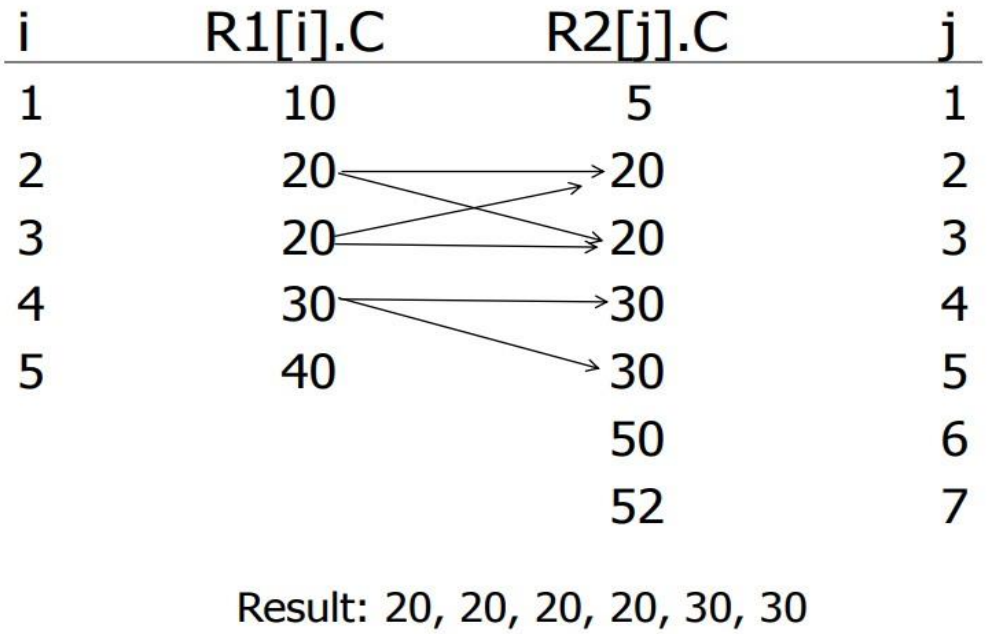
Read chunk: 100 IOs (100 blocks)

Read R1: 1000 IOs (1000 blocks)

1,100

Total= 5 chunks x 1,100 = 5,500 IOs

Merge Join (Sort-based Join Algorithms)



Result: 20, 20, 20, 20, 30, 30

- Smarter while working with two columns involved in the join.
- Assumption: two columns are sorted from smallest to largest.
- Have pointers i and j.
- When i and j not pointing to same value; I will advance both.
- When same value in both sides; I will advance the j till get a different new value in j, so will advance the smaller counter

Merge join (pseudo-code)

(1) if R1 and R2 not sorted, sort them

(2) $i \leftarrow 1$; $j \leftarrow 1$;

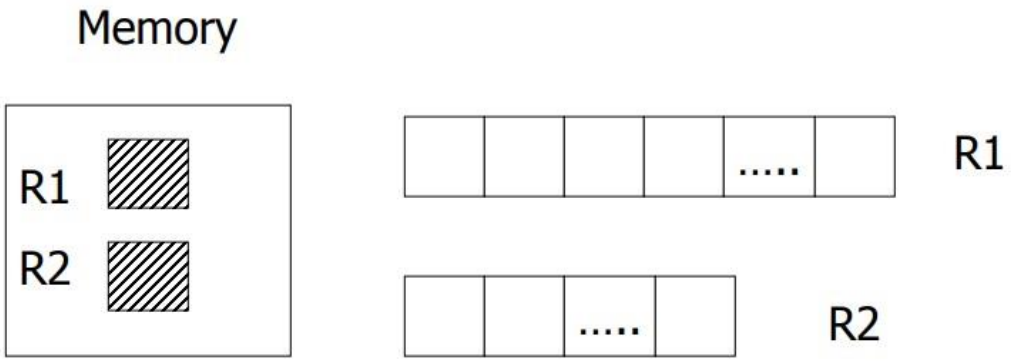
```
While (i ≤ T(R1)) ∧ (j ≤ T(R2)) do
    if R1[ i ].C = R2[j].C then
            outputTuples
    else
        if R1[ i ].C > R2[ j ].C then
            j ← j+1
        else
        if R1[ i ].C < R2[ j ].C then
            i ← i+1
```

```
While (R1[ i ].C = R2[ j ].C) ∧ (i ≤ T(R1)) do
    jj ← j;
    while ((jj ≤ T(R2) ∧ (R1 [i ].C = R2[ jj ].C)) do
            output pair R1[ i ], R2[ jj ];
            jj ← jj+1
    i ← i+1
```

Merge Join

- Recall { T(R1) = 10,000 T(R2) = 5,000
  S(R1) = S(R2) =1/10 block
  MEM=101 blocks

- Both R1, R2 ordered by C; relations contiguous



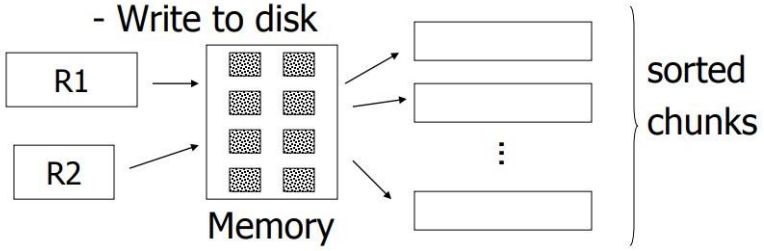
Total cost: Read R1 cost + read R2 cost
= 1000 + 500 = 1,500 IOs

Merge Join

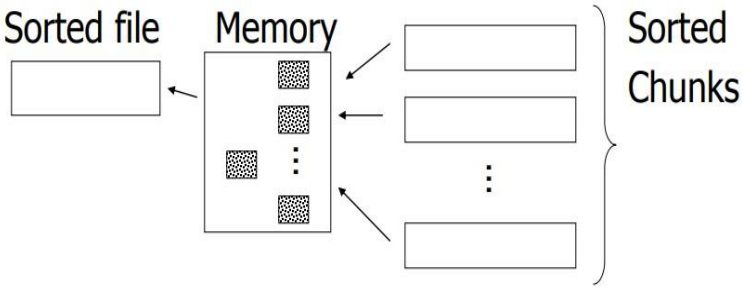
R1, R2 both are not sorted, but contiguous. We will need to Sort R1, R2 first
Divide to two phases to sort.

(i) For each 100 blk chunk of R:

- Read chunk
- Sort in memory
- Write to disk



(ii) Read all chunks + merge + write out



- Load again the pages that are sorted but will load the pages that have an overlap in the ranges (need to keep track of min and max of each page).
- Merge them and write back to memory.

Merge Join Cost

Cost: Sort

Each tuple is read,written,
read, written

so...

Sort cost R1: 4 x 1,000 = 4,000

Sort cost R2: 4 x 500 = 2,000

Total cost = sort cost + join cost
= 6,000 + 1,500 = 7,500 IOs

Join with index

For each r ∈ R1 do

  [X ← index (R2, C, r.C)

    for each s ∈ X do

      output r,s pair]

Assume R2.C is indexed

Note: X ← index(rel, attr, value)

then X = set of rel tuples with attr = value

- Assume R1.C is the indexed column.
- Assume R2 is a contiguous, unordered relation.
- Assume that the index of R1.C fits in the memory.

Cost: Reads: 500 IOs (for R2)

for each R2 tuple:

- probe R1.C index: free (0 IO)
- if match, read R1.C tuple: 1 IO

Number of matching tuples

Case 1: R1.C is key, R2.C is foreign key
then expect = 1 tuple to match

Total cost = 500+5000(1)1+0 = 5,500 IO

- 500 (reading all R2) + 5000 (number of rows R2 to get the match after checking the R1.C index to load the page containing the match) + 0 (cost of index).

Case 2: R2.C references R1.C
R1.C contain duplicates

say V(R1,C) = 5000, T(R1) = 10,000
with uniform assumption expect =
10,000/5,000 = 2 tuples to match

Total cost = 500+5000(2)1+0 = 10,500 IO

case 3: R2.C references R1.C
R1.C contain duplicates + no stats
Say DOM(R1, C)=1,000,000
T(R1) = 10,000
with alternate assumption

$$\text{Expect} = \frac{10{,}000}{1{,}000{,}000} = \frac{1}{100}$$

Total cost = 500+5000(1/100)1+0=550 IO

What if index doesn't fit in memory

Example: say R1.C index is 201 blocks

- Keep root + 99 leaf nodes in memory
- Expected cost of each probe is

$$E = (0)\frac{99}{200} + (1)\frac{101}{200} \approx 0.5$$

For case 1:

= 500+5000 [0.5+ 1]

= 500+5000 [1.5]

= 500+7,500 = 8000 IO

500 (reading R2) +5000 (going to the index and with every probe in the index we went and read a page (1) + add half an I/O on average to load the B+tree nodes to put them in memory)

For case 2:

= 500+5000 [Probe + get records]

= 500+5000 [0.5+2]

= 500+12,500 = 13,000

For case 3:

= 500+5000[0.5 × 1 + (1/100) × 1]

= 500+2500+50 = 3050 IOs

B+ Tree execution

With every SQL statement execution, what is the cost of working with B+tree ?

- If it fits in memory, then no cost.
- If part exists in memory, then we need to load the rest (probability problem, chance to be in memory).
- Like in the previous example, the probability is half.
- Half the time we will go to disk to load the page, and half the time we won’t.

Hash Join

- Hash function h, range $0 \rightarrow k$
- Buckets for R1: $G_0, G_1, \ldots G_k$
- Buckets for R2: $H_0, H_1, \ldots H_k$

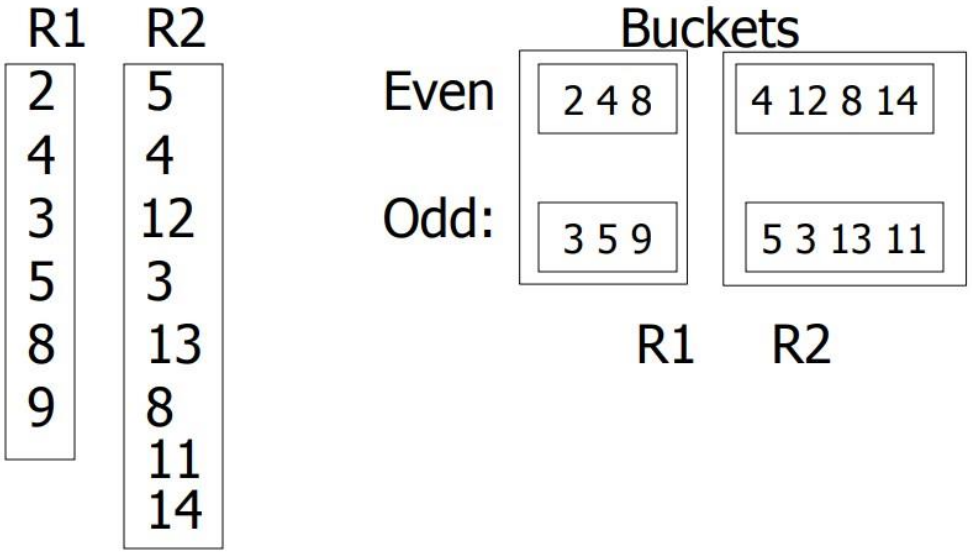
Algorithm

(1) Hash R1 tuples into G buckets

(2) Hash R2 tuples into H buckets

(3) For i = 0 to k do

match tuples in G_i, H_i buckets



Hash function:
If value is even:
put it in a bucket.
Else:
put it in another bucket.

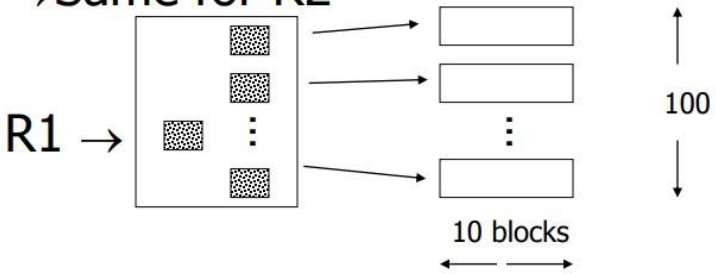
Hash Join

- R1, R2 contiguous (un-ordered)

→ Use 100 buckets

→Read R1, hash, + write buckets

→Same for R2



"Bucketize:" Read R1 + write

Read R2 + write

Join: Read R1, R2 (to get the actual rows)

Total cost = 3 x [1000+500] = 4500